OPMIG-08: Security, Masking & Compliance

> **Series:** OPMIG — OpenPipeline Migration | **Notebook:** 8 of 10 | **Created:** December 2025 | **Last Updated:** 10/30/2026

Table of Contents

1. Understanding Masking
2. Built-in Masking Patterns
3. Custom Masking with replacePattern
4. Compliance Patterns
5. Field Removal for Security
6. Validating Masking
7. Testing Masking in DPL Architect
8. Best Practices
9. Complete Security Pipeline Example

Learning Objectives

By completing this notebook, you will:

1. Configure masking processors for PII protection
2. Use built-in and custom masking patterns
3. ⭐ **NEW:** Implement GDPR compliance checklist
4. ⭐ **NEW:** Implement HIPAA compliance checklist
5. ⭐ **NEW:** Implement PCI-DSS compliance checklist
6. ⭐ **NEW:** Implement SOC 2 compliance checklist

7. Validate masking effectiveness
8. Design complete security pipelines

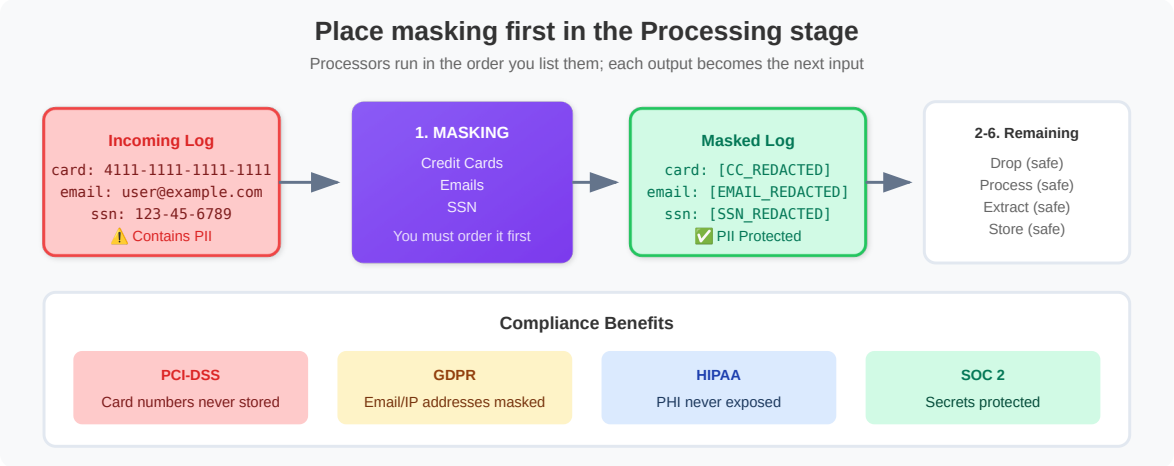
Prerequisites

Requirement	Details
Dynatrace Environment	Dynatrace SaaS with Grail and OpenPipeline access — Managed is not covered by this series
Permissions	`settings:read` and `settings:write` (OpenPipeline configuration)
API Access	`logs.read` token scope
DPL Architect	Access to `https://{env}.apps.dynatrace.com/ui/apps/dynatrace.dpl.architect`
Knowledge	OPMIG-01 through OPMIG-07; understanding of compliance requirements (PCI, HIPAA, GDPR)

Security Stage Overview

Place masking processors **first within the Processing stage**, before any processor that copies or parses the sensitive field. Masking is not automatic: a processor listed ahead of the masking processor sees the raw value.

Routing runs before any pipeline processor, so routing matchers always see the raw value — keep PII out of routing conditions, or mask at capture (OneAgent-side masking) when the value must never reach Dynatrace.



Why Masking First?

Order	Reason
Before parsing	Sensitive data never extracted to fields
Not before routing	Routing precedes every pipeline processor — keep PII out of routing conditions
Before storage	Compliance-safe from the start
Before extraction	Metrics/events don't contain PII

> **Sources:** Data flow (DT docs) — *"After data is ingested (and optionally pre-processed), it's routed to pipelines."*; Processing in OpenPipeline (DT docs) — *"The processor order in the stage; each processor output becomes the input for the next one."*

Understanding Masking

What Masking Does

Masking replaces sensitive patterns with redacted values:

Before:

```
User email: john.doe@example.com, card: 4111-1111-1111-1111
```

After:

```
User email: [EMAIL_REDACTED], card: [CC_REDACTED]
```

Masking with a DQL processor

Masking is done with a **DQL** processor (`replacePattern`, or `fieldsRemove` for whole fields); there is no dedicated masking processor type. A masking processor is configured as:

Field	Description
Processor type	DQL
Name	Descriptive name for the mask
Matching condition	When to apply masking (`true` for all records)
Definition	One `fieldsAdd <field> = replacePattern(<field>, "<DPL pattern>", replacement: "<text>")` per field to mask, joined with `|`

> **Sources:** Processing in OpenPipeline (DT docs) — *"The following table lists alphabetically all available processors in a pipeline."* **Derived:** none of the processors in that table is a masking type; DQL is the one that runs `replacePattern`.

Masking vs. Dropping

Action	Use When
Mask	Need the record, just hide sensitive parts
Drop	Entire record is not needed

Built-in Masking Patterns

OpenPipeline includes pre-built matchers for common sensitive data.

Credit Card Numbers

DPL Pattern:

```
!&lt;&lt;[0-9A-Za-z-] CREDITCARD !&gt;&gt;[0-9A-Za-z-]
```

Matches:

- `4111111111111111`
- `4111-1111-1111-1111`
- `4111 1111 1111 1111`

Masking Processor (DQL):

```
// Name: Mask Credit Cards · Matching condition: true
fieldsAdd content = replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] CREDITCARD
!&gt;&gt;[0-9A-Za-z-]", replacement: "[CC_REDACTED]")
| fieldsAdd message = replacePattern(message, "!&lt;&lt;[0-9A-Za-z-]
CREDITCARD !&gt;&gt;[0-9A-Za-z-]", replacement: "[CC_REDACTED]")
```

> **Bound `CREDITCARD`.** On its own, the matcher finds a Luhn-valid digit run wherever it sits, including inside a UUID: `req 9cbcb510-8833-4981-87ab-1234567890ab ok` becomes `req 9cbcb[CC_REDACTED]ab-1234567890ab ok`. On a validation tenant, 1,366 of the 1,541 log records a bare `CREDITCARD` changed in 15 minutes were IDs, not card numbers (10/12/2026), and an ingest processor would have corrupted every one of them, forward-only. The negative lookbehind `!<<` and lookahead `!>>` reject a match that touches a letter, digit or hyphen. The UUID is then left intact, and the three formats above, plus `x=4111111111111111,y`, are still masked (verified 10/12/2026).

Email Addresses

> **There is no built-in `EMAIL` matcher in DPL.** A pattern of `EMAIL` is rejected with `Named pattern element 'EMAIL' is not valid`, so a masking processor built on it

> never
> redacts anything. Use the hand-rolled character-class pattern below.

DPL Pattern:

```
[a-zA-Z0-9._%+-]+ '@' [a-zA-Z0-9.-]+
```

Matches:

- `user@example.com`
- `first.last@company.org`
- `a-b_c%d+e@sub.domain.co.uk`

Masking Processor (DQL):

```
// Name: Mask Emails · Matching condition: true
fieldsAdd content = replacePattern(content, "[a-zA-Z0-9._%+-]+ '@' [a-zA-Z0-9.-]+", replacement: "[EMAIL_REDACTED]")
| fieldsAdd message = replacePattern(message, "[a-zA-Z0-9._%+-]+ '@' [a-zA-Z0-9.-]+", replacement: "[EMAIL_REDACTED]")
```

> **Note:** the domain class includes `.`, so a sentence-final period is absorbed into the match (`mail x@y.io.` → `mail [EMAIL_REDACTED]`). That over-masks by one character rather
> than under-masking — the safe direction for PII.

IP Addresses

DPL Pattern:

```
IPADDR
IPV4ADDR
IPV6ADDR
```

Masking Processor (DQL):

```
// Name: Mask IP Addresses · Matching condition: true
fieldsAdd content = replacePattern(content, "IPADDR", replacement: "
[IP_REDACTED]")
| fieldsAdd client_ip = replacePattern(client_ip, "IPADDR", replacement: "
[IP_REDACTED]")
| fieldsAdd remote_addr = replacePattern(remote_addr, "IPADDR", replacement: "
[IP_REDACTED]")
```

Phone Numbers (Custom Pattern)

DPL Pattern:

```
'(' [0-9]{3} ')' SPACE? [0-9]{3} '-' [0-9]{4}
```

> **`INT` does not accept a `{n}` quantifier** — it supports only `*` and `+`, and `INT{3}` fails with `INT only supports '*' (nullable) and '+' (not nullable) quantifiers`.
> Use a `[0-9]{n}` character class for fixed-width digit runs.

Matches:

- `(555) 123-4567`
- `(555)123-4567`

Custom Masking with replacePattern

The `replacePattern` function enables custom masking using DPL patterns.

Syntax

```
fieldsAdd content = replacePattern(content, "PATTERN", replacement:
"REPLACEMENT")
```

Example: Mask SSN

Pattern: SSN format `123-45-6789`

```
fieldsAdd content = replacePattern(content, "[0-9]{3} '-' [0-9]{2} '-' [0-9]
{4}", replacement: "[SSN_REDACTED]")
```

Example: Mask API Keys

Pattern: Keys starting with `key_`, masked through to the next whitespace

```
fieldsAdd content = replacePattern(content, "'key_' NSPACE", replacement: "
[API_KEY_REDACTED]")
```

> **Why not `'key_' WORD`?** `WORD` matches a greedy run of alphanumerics and underscores,
> but stops at `-`. Against `key_XY-99` it yields `[API_KEY_REDACTED]-99`, leaving part of the secret in the log. `NSPACE` consumes to the next whitespace and masks the whole token.

Example: Mask Bearer Tokens

Pattern: Bearer tokens in Authorization headers

```
fieldsAdd content = replacePattern(content, "'Bearer ' NSPACE", replacement:
"Bearer [TOKEN_REDACTED]")
```

Example: Mask Passwords in URLs

Pattern: Password parameter in query strings

```
fieldsAdd content = replacePattern(content, "'password=' [^&amp;\\s]+",
replacement: "password=[REDACTED]")
```

> **Why not `'password=' LD:pwd ('&'|EOL)`?** It only masks when another parameter follows. On `…?user=bob&password=hunter2`, with the password last, it matched nothing and the password was **stored in clear** (verified 10/02/2026). The DPL character class `[^&\s]+` (written `\\s` inside the DQL string) stops at the next `&` or whitespace and masks both cases.

Chaining Multiple Masks

```
// Apply multiple masks in sequence
fieldsAdd content = replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] CREDITCARD
!&gt;&gt;[0-9A-Za-z-]", replacement: "[CC_REDACTED]")
| fieldsAdd content = replacePattern(content, "[a-zA-Z0-9._%+-]+ '@' [a-zA-Z0-
9.-]+", replacement: "[EMAIL_REDACTED]")
| fieldsAdd content = replacePattern(content, "IPADDR", replacement: "
[IP_REDACTED]")
```

Compliance Patterns

> **End a key=value mask with `NSPACE`, not `LD`.** A trailing `LD` runs to the end of the line, so `'userId=' LD:uid` turns `userId=u-77 action=view status=OK` into `userId=[USER_REDACTED]` — everything after the key is destroyed, including fields you parse later. `NSPACE` stops at the next space (verified 10/02/2026). If a value can contain spaces, close the pattern with its delimiter instead.

PCI-DSS Compliance

Requirements:
- Mask all Primary Account Numbers (PANs)
- Mask CVV/CVC codes
- Mask cardholder names when combined with PANs

Pipeline Configuration:

```
// Mask credit card numbers
fieldsAdd content = replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] CREDITCARD
!&gt;&gt;[0-9A-Za-z-]", replacement: "[PAN_REDACTED]")
// Mask CVV (3-4 digits after 'cvv=' or 'cvc=')
| fieldsAdd content = replacePattern(content, "('cvv='|'cvc='|'CVV='|'CVC=')
[0-9]{3,4}", replacement: "cvv=[REDACTED]")
```

GDPR Compliance

Requirements:

- Mask personal identifiers (emails, phone numbers)
- Mask IP addresses (considered PII in EU)
- Mask names when identifiable

Pipeline Configuration:

```
// Mask emails
fieldsAdd content = replacePattern(content, "[a-zA-Z0-9._%+-]+ '@' [a-zA-Z0-
9.-]+", replacement: "[PII_REDACTED]")
// Mask IP addresses
| fieldsAdd content = replacePattern(content, "IPADDR", replacement: "
[IP_REDACTED]")
// Mask user IDs
| fieldsAdd content = replacePattern(content, "'userId=' NSPACE", replacement:
"userId=[USER_REDACTED]")
```

HIPAA Compliance

Requirements:

- Mask Protected Health Information (PHI)
- Mask patient identifiers
- Mask medical record numbers

Pipeline Configuration:

```
// Mask patient IDs
fieldsAdd content = replacePattern(content, "'patientId=' NSPACE",
replacement: "patientId=[PHI_REDACTED]")
// Mask MRNs
| fieldsAdd content = replacePattern(content, "'mrn=' NSPACE", replacement:
"mrn=[PHI_REDACTED]")
// Mask SSNs
| fieldsAdd content = replacePattern(content, "[0-9]{3} '-' [0-9]{2} '-' [0-9]
{4}", replacement: "[SSN_REDACTED]")
```

SOC 2 Compliance

Requirements:

- Mask credentials and secrets

- Mask authentication tokens
- Audit access to sensitive data

Pipeline Configuration:

```
// Mask passwords. DPL literals are case-sensitive, so list each casing;
&lt;&lt;(…) keeps the key name
fieldsAdd content = replacePattern(content, "&lt;&lt;(('password' | 'PASSWORD'
| 'Password' | 'passwd' | 'pwd') '=') NSPACE", replacement: "[REDACTED]")
// Mask API keys and tokens (token= also covers access_token=)
| fieldsAdd content = replacePattern(content, "&lt;&lt;(('api_key' | 'API_KEY'
| 'apiKey' | 'apikey' | 'APIKEY' | 'token' | 'TOKEN') '=') NSPACE",
replacement: "[REDACTED]")
// Mask JSON-quoted secrets: "password":"…", "api_key": "…"
| fieldsAdd content = replacePattern(content, "&lt;&lt;('\"' ('password' |
'passwd' | 'api_key' | 'apiKey' | 'apikey' | 'token' | 'secret') '\"' ' '* ':'
' '* '\"') [^\"]+", replacement: "[REDACTED]")
// Mask tokens
| fieldsAdd content = replacePattern(content, "'Bearer ' NSPACE", replacement:
"Bearer [REDACTED]")
```

> **There is no case-insensitive literal in DPL.** The SOC 2 key list used to cover `api_key=`, `apiKey=` and `API_KEY=` only, so `apikey=`, `token=`, `PASSWORD=` and JSON-quoted keys such as `{"password":"hunter2"}` were stored in clear. The statements above mask all of them and keep each key name; `monkey=banana` is left alone (verified 10/12/2026 with `data record(...)`). List every casing your applications emit, and keep a separate statement for JSON-quoted keys. A sequence inside an alternation needs its own parentheses: `<<(('token' | 'TOKEN') '=')`, not `<<('token' '=' | …)`.

Field Removal for Security

Sometimes it's better to remove entire fields rather than mask values.

Using fieldsRemove

```
// Remove sensitive fields entirely
fieldsRemove password, secret, api_key, token, authorization
```

When to Remove vs. Mask

Scenario	Action
Field always contains sensitive data	Remove
Only some values are sensitive	Mask
Need to know field existed	Mask with placeholder
Compliance requires no trace	Remove

Conditional Field Removal

Apply removal only when field contains sensitive patterns:

```
// Remove field only if it contains a pattern
fieldsAdd authorization = if(contains(authorization, "Bearer"), null, else:
authorization)
```

Validating Masking

After configuring masking, verify it's working correctly.

> ⚠️ **Important:** Test with sample data before deploying to production.

```
// Check for any remaining card numbers in stored logs — if masking works,
this returns 0 rows.
// replacePattern() finds a CREDITCARD match anywhere; matchesPhrase(content,
"4111") would not,
// because an unbroken 16-digit number is a single token that "4111" never
equals.
// The !&lt;&lt; / !&gt;&gt; bounds keep digit runs inside UUIDs from counting
as cards.
fetch logs, from: now() - 24h
| filter replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] CREDITCARD !&gt;&gt;
[0-9A-Za-z-]", replacement: "") != content
| fields timestamp, log.source, dt.openpipeline.pipelines
| limit 10
```

```
// Check for any remaining email addresses — if masking works, this returns 0
rows.
// Uses the same DPL pattern as the masking processor, so it finds any
address, not a fixed domain list.
fetch logs, from: now() - 24h
| filter replacePattern(content, "[a-zA-Z0-9._%+-]+ '@' [a-zA-Z0-9.-]+",
replacement: "") != content
| fields timestamp, log.source, dt.openpipeline.pipelines
| limit 10
```

```
// Verify redaction placeholders are present
// This confirms masking is actively working
fetch logs, from: now() - 24h
| filter contains(content, "[REDACTED]")
   OR contains(content, "[CC_REDACTED]")
   OR contains(content, "[EMAIL_REDACTED]")
   OR contains(content, "[PII_REDACTED]")
| summarize {masked_count = count()}, by: {dt.openpipeline.pipelines}
| sort masked_count desc
```

```
// Sample masked logs to verify format
fetch logs, from: now() - 24h
| filter contains(content, "REDACTED")
| fields timestamp, content
| limit 20
```

```
// Audit: Count masked records by pipeline and type
fetch logs, from: now() - 24h
| fieldsAdd has_cc_mask = contains(content, "[CC_REDACTED]"),
           has_email_mask = contains(content, "[EMAIL_REDACTED]"),
           has_ip_mask = contains(content, "[IP_REDACTED]"),
           has_pii_mask = contains(content, "[PII_REDACTED]")
| summarize {
    total = count(),
    cc_masked = countIf(has_cc_mask),
    email_masked = countIf(has_email_mask),
    ip_masked = countIf(has_ip_mask),
    pii_masked = countIf(has_pii_mask)
  }, by: {dt.openpipeline.pipelines}
```

```
// Check for SSN-shaped values (###-##-####) that masking missed — should
return 0 rows
fetch logs, from: now() - 24h
| filter replacePattern(content, "[0-9]{3} '-' [0-9]{2} '-' [0-9]{4}",
replacement: "") != content
| fields timestamp, log.source, dt.openpipeline.pipelines
| limit 50
```

Testing Masking in DPL Architect

Before deploying masking rules, test them:

1. Open **DPL Architect** in Dynatrace
2. Paste sample log with sensitive data
3. Test your DPL pattern matches correctly
4. Verify replacement produces expected output

Test Cases to Include

Test Case	Input	Expected Output
Credit card	`4111-1111-1111-1111`	`[CC_REDACTED]`
Credit card no dashes	`4111111111111111`	`[CC_REDACTED]`
Email	`user@example.com`	`[EMAIL_REDACTED]`
IPv4	`192.168.1.1`	`[IP_REDACTED]`
IPv6	`2001:db8::1`	`[IP_REDACTED]`
Mixed content	Multiple patterns	All redacted

Best Practices

Masking Configuration

Practice	Reason
Keep PII out of routing conditions	Routing runs before any pipeline processor, so routing matchers see the raw value
Use descriptive placeholders	Aids troubleshooting
Test patterns thoroughly	Avoid over/under masking
Document masking rules	Compliance audits

Pattern Design

Practice	Reason
Be specific	Avoid false positives
Handle variations	Same data, different formats
Use built-in matchers	Pre-tested, reliable
Chain patterns	Cover all sensitive data

Compliance

Practice	Reason
Document all masking	Audit trail
Regular pattern review	New data sources
Test with real samples	Ensure effectiveness
Monitor for leaks	Continuous validation

Performance

Practice	Reason
Apply to specific fields	Faster than all fields
Use matching conditions	Skip unneeded records
Order patterns by frequency	Most common first

Complete Security Pipeline Example

Pipeline: `payment-logs-secure`

Masking Processor 1: Credit Cards (DQL processor, matching condition `true`)

```
fieldsAdd content = replacePattern(content, "!&lt;&lt;[0-9A-Za-z-] CREDITCARD
!&gt;&gt;[0-9A-Za-z-]", replacement: "[CC_REDACTED]")
| fieldsAdd card_number = replacePattern(card_number, "CREDITCARD",
replacement: "[CC_REDACTED]")
```

Masking Processor 2: CVV Codes

```
fieldsAdd content = replacePattern(content, "('cvv='|'cvc='|'CVV='|'CVC=') [0-
9]{3,4}", replacement: "cvv=[REDACTED]")
```

Masking Processor 3: Emails (DQL processor, matching condition `true`)

```
fieldsAdd content = replacePattern(content, "[a-zA-Z0-9._%+-]+ '@' [a-zA-Z0-
9.-]+", replacement: "[EMAIL_REDACTED]")
| fieldsAdd customer_email = replacePattern(customer_email, "[a-zA-Z0-9._%+-]+
'@' [a-zA-Z0-9.-]+", replacement: "[EMAIL_REDACTED]")
```

Masking Processor 4: IP Addresses (DQL processor, matching condition `true`)

```
fieldsAdd content = replacePattern(content, "IPADDR", replacement: "
[IP_REDACTED]")
| fieldsAdd client_ip = replacePattern(client_ip, "IPADDR", replacement: "
[IP_REDACTED]")
| fieldsAdd remote_addr = replacePattern(remote_addr, "IPADDR", replacement: "
[IP_REDACTED]")
```

Masking Processor 5: Auth Tokens

```
fieldsAdd content = replacePattern(content, "'Bearer ' NSPACE", replacement:
"Bearer [TOKEN_REDACTED]")
| fieldsRemove authorization, auth_token
```

Pipeline Verification Query

```
// Verify complete masking for payment-logs pipeline
fetch logs, from: now() - 1h
// dt.openpipeline.pipelines is an array of "&lt;scope&gt;:&lt;pipeline
id&gt;" strings, so == never matches
| filter matchesValue(dt.openpipeline.pipelines, "*payment-logs-secure*")
| summarize {
    total = count(),
    with_cc = countIf(contains(content, "[CC_REDACTED]")),
    with_email = countIf(contains(content, "[EMAIL_REDACTED]")),
    with_ip = countIf(contains(content, "[IP_REDACTED]")),
    with_token = countIf(contains(content, "[TOKEN_REDACTED]"))
  }
```

Next Steps

Now that security is configured, complete your migration:

Notebook	Focus Area
OPMIG-09	Troubleshooting & Validation

References

- OpenPipeline Masking
- DPL replacePattern
- Data Privacy in Dynatrace
- Compliance Best Practices

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.